Guide Complet : Comprendre les bases de Jenkins

Ce document a été conçu pour accompagner les débutants dans leur apprentissage de Jenkins. Il couvre de manière détaillée tous les concepts fondamentaux : architecture, interface, jobs, pipelines, agents, credentials, outils et variables d'environnement. Que Jenkins soit installé directement sur un serveur Linux ou via un conteneur Docker, ce guide s'applique dans les deux cas — le fonctionnement interne de Jenkins est identique.

> ***Référence aux TPs :*** *Ce document complète les Travaux Pratiques du cours CI/CD. Vous trouverez des renvois vers les TPs correspondants tout au long du texte.*

Table des matières

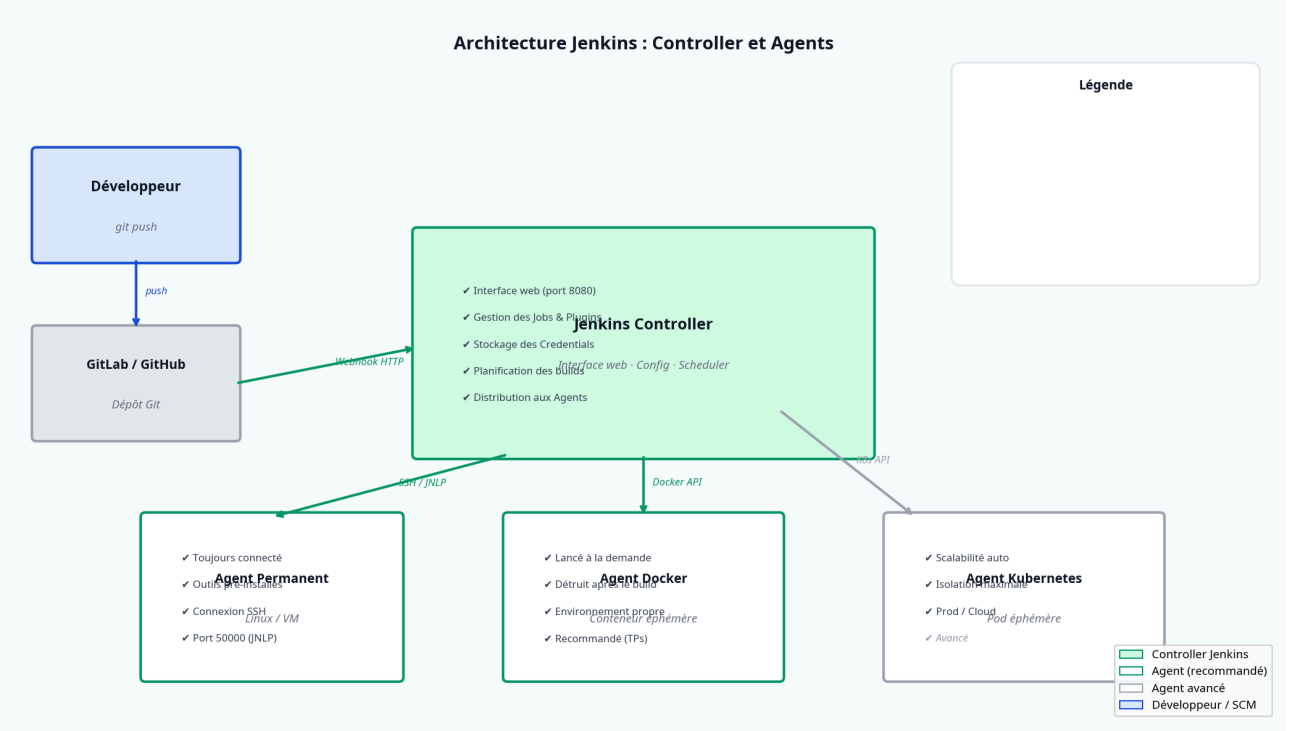
1. L'Architecture de Jenkins
2. L'Interface Web et la Navigation
3. Les Jobs Freestyle
4. Les Pipelines (Pipeline as Code)
5. Les Agents
6. Les Outils Globaux
7. Les Variables d'Environnement
8. Les Credentials
9. Les Plugins
10. Le Workspace et le Nettoyage
11. Les Notifications
12. Bonnes pratiques et pièges courants
13. FAQ Débutant

---

1. L'Architecture de Jenkins

Vue d'ensemble

La première chose à comprendre avec Jenkins est son architecture distribuée. Jenkins est conçu pour fonctionner avec deux types de composants : un **Controller** (le cerveau) et des **Agents** (les ouvriers). Cette séparation est fondamentale pour comprendre comment Jenkins fonctionne.



Le Controller (anciennement appelé Master)

Le **Controller** est l'instance principale de Jenkins. C'est lui que vous démarrez, que vous configurez, et auquel vous accédez via votre navigateur web (généralement sur le port `8080`).

Ses responsabilités sont :

- Héberger l'**interface utilisateur web** permettant de créer et surveiller les jobs.
- Stocker toute la **configuration** : jobs, plugins, credentials, paramètres globaux.
- **Planifier et déclencher** les builds (exécutions) en réponse à des événements (push Git, planification cron, déclenchement manuel).
- **Distribuer le travail** aux Agents disponibles.

Règle d'or : Il ne faut **jamais** exécuter de builds lourds (compilation, tests, analyse de code) directement sur le Controller. Si le Controller manque de ressources à cause d'un build, toute l'interface Jenkins devient indisponible pour tous les utilisateurs. Le Controller doit rester léger et disponible.

Les Agents (anciennement Slaves ou Nodes)

Les **Agents** sont les machines qui exécutent concrètement les tâches. Le Controller leur délègue le travail. Un Agent peut être :

- Une **machine virtuelle ou physique Linux** configurée pour écouter le Controller en permanence (Agent permanent).
- Un **conteneur Docker** lancé à la volée uniquement pour la durée d'un build, puis détruit (Agent éphémère). C'est la méthode recommandée et celle utilisée dans les TPs (TP 2, 3, 4, 5).
- Un **pod Kubernetes** pour les environnements cloud à grande échelle (usage avancé).

Communication Controller ↔ Agent

Le Controller et les Agents communiquent de deux façons principales :

Méthode	Description	Cas d'usage
SSH	Le Controller se connecte à l'Agent via SSH avec une clé privée.	Agents Linux permanents.
JNLP / Inbound	L'Agent initie la connexion vers le Controller (port 50000).	Agents derrière un pare-feu, agents Windows.

---

2. L'Interface Web et la Navigation

Le Tableau de bord (Dashboard)

Lorsque vous vous connectez à Jenkins, vous arrivez sur le **Tableau de bord**. Il liste tous vos projets (appelés *Jobs*). À côté de chaque Job, deux indicateurs visuels vous informent immédiatement de son état :

La boule de statut (S — Status) : Indique l'état du dernier build exécuté.

Couleur	Signification
🔵 Bleu	Succès — le dernier build s'est terminé sans erreur.
🔴 Rouge	Échec — le dernier build a échoué (erreur de compilation, test raté…).
🟡 Jaune	Instable — le build a réussi mais des tests ont échoué.
⚪ Gris	Jamais exécuté — le job vient d'être créé.

L'icône météo (W — Weather) : Représente la tendance des derniers builds. Un soleil signifie que les derniers builds ont tous réussi. Un orage signifie que la majorité ont échoué. Cela donne une vue rapide de la "santé" d'un projet dans le temps.

La vue d'un Job

En cliquant sur un Job, vous accédez à sa page de détail. Vous y trouvez :

- **L'historique des builds** : liste numérotée de toutes les exécutions passées avec leur statut et leur durée.
- **Console Output** : les logs complets d'un build. C'est ici que vous trouverez la cause d'un échec.
- **Workspace** : les fichiers présents sur l'Agent après le dernier build.
- **Configurer** : accès au formulaire de configuration du Job.

Administrer Jenkins (Manage Jenkins)

Le menu **"Administrer Jenkins"** (ou *Manage Jenkins*) est la tour de contrôle de votre instance. Vous y trouverez les sections suivantes, que nous détaillerons dans ce document :

Section	Rôle
System	Configuration globale (URL de Jenkins, variables d'environnement globales, notifications).
Tools	Configuration des outils installés sur les agents (Maven, JDK, Git, Docker).
Plugins	Installation, mise à jour et suppression des plugins.
Nodes and Clouds	Gestion des Agents (ajout, suppression, surveillance).
Credentials	Stockage sécurisé des mots de passe, tokens et clés SSH.
Security	Gestion des utilisateurs, des rôles et des permissions.

---

3. Les Jobs Freestyle

Le **Job Freestyle** est la méthode historique et la plus simple pour créer une tâche dans Jenkins. Tout se configure via des formulaires graphiques, sans écrire de code. C'est un excellent point de départ pour comprendre les concepts de base.

Créer un Job Freestyle

Depuis le tableau de bord, cliquez sur **"Nouveau Item"** (ou *New Item*), donnez un nom à votre job, sélectionnez **"Projet Freestyle"** et cliquez sur OK.

Les sections de configuration

1. General Nom, description du job. Vous pouvez également cocher *"Ce build est paramétré"* pour demander des informations à l'utilisateur avant chaque exécution (ex: un numéro de version, un environnement cible).

2. Source Code Management (Gestion du code source) C'est ici que vous indiquez à Jenkins où récupérer le code source. En choisissant **Git**, vous renseignez :

- L'URL du dépôt (ex: `https://gitlab.com/mon-organisation/mon-projet.git`).
- Les credentials pour accéder au dépôt privé (sélectionnés depuis le Credentials Store).

- La branche à construire (ex: `*/main` ou `*/develop`).

3. Build Triggers (Déclencheurs) Quand le job doit-il s'exécuter ?

Déclencheur	Description	Recommandation
Webhook	GitLab/GitHub notifie Jenkins instantanément à chaque push.	✅ Recommandé en production.
Poll SCM	Jenkins vérifie Git toutes les X minutes. Syntaxe cron : `H/5 * * * *`	Acceptable si webhook impossible.
Build periodically	Exécution à heure fixe (ex: build nocturne). Syntaxe cron : `0 2 * * *`	Pour les builds planifiés.
Déclenchement manuel	L'utilisateur clique sur "Lancer un build".	Pour les tests ponctuels.

4. Build Steps (Étapes de build) Les actions concrètes à exécuter. Pour un projet Java, vous ajouterez un step **"Exécuter un script shell"** :

```
mvn clean package -B
```

5. Post-build Actions (Actions post-build) Ce qui se passe après le build :

- **Archiver les artefacts** : conserver le fichier `.jar` produit.
- **Publier les résultats JUnit** : afficher les résultats des tests dans Jenkins.
- **Envoyer un email** : notifier l'équipe en cas d'échec.

Limites du Freestyle

Le Job Freestyle présente des inconvénients majeurs dans un contexte professionnel :

- La configuration est stockée dans Jenkins, **pas dans Git**. Si Jenkins plante, la configuration est perdue.
- Il n'y a **pas d'historique des modifications** de la configuration (pas de versionnement).
- Il est **difficile à maintenir** sur des dizaines ou centaines de projets.

C’est pourquoi les **Pipelines** ont été créés.

---

4. Les Pipelines (Pipeline as Code)

Le concept fondamental

Le **Pipeline as Code** consiste à décrire l’intégralité du processus de build dans un fichier texte appelé `Jenkinsfile`, stocké à la racine de votre dépôt Git, avec votre code source.

Concrètement, cela signifie que si vous modifiez votre processus de build (ex: vous ajoutez une étape d’analyse SonarQube), vous faites un commit dans Git. Ce changement est tracé, réversible, et peut être relu par d’autres développeurs via une Merge Request.

Les deux types de Jobs Pipeline

Type	Description	Cas d’usage
Pipeline	Job unique associé à une branche fixe.	Projets simples, scripts d’administration.
Multibranch Pipeline	Scanne automatiquement toutes les branches du dépôt et crée un job par branche contenant un `Jenkinsfile`.	Projets avec GitFlow (main, develop, feature/*).

La syntaxe Déclarative

Il existe deux syntaxes pour écrire un Jenkinsfile : **Scripted** (Groovy pur, très flexible mais complexe) et **Déclarative** (structure imposée, plus lisible). La syntaxe Déclarative est la recommandée pour les nouveaux projets.

Voici la structure complète d’un Jenkinsfile déclaratif, commentée :

```
pipeline {

    // -------------------------------------------------------
    // AGENT : sur quelle machine ce pipeline s'exécute-t-il ?
    // -------------------------------------------------------
    agent {
        docker {
            image 'maven:3.9-eclipse-temurin-17'
            args '-v /root/.m2:/root/.m2'
        }
    }

    // -------------------------------------------------------
    // ENVIRONMENT : variables disponibles dans tout le pipeline
    // -------------------------------------------------------
    environment {
        APP_NAME    = 'demo-java-app'
        APP_VERSION = "1.0.${BUILD_NUMBER}"
    }

    // -------------------------------------------------------
    // STAGES : les grandes étapes du pipeline
    // -------------------------------------------------------
    stages {

        stage('Checkout') {
            steps {
                // Récupérer le code depuis le dépôt Git configuré dans Jenkins
                checkout scm
            }
        }

        stage('Compilation') {
            steps {
                sh 'mvn clean compile -B'
            }
        }

        stage('Tests Unitaires') {
            steps {
                sh 'mvn test -B'
            }
            post {
                // Publier les résultats JUnit après cette étape, qu'elle
réussisse ou non
                always {
```

```
                    junit 'target/surefire-reports/*.xml'
                }
            }
        }

        stage('Packaging') {
            steps {
                sh 'mvn package -DskipTests -B'
                // Archiver le JAR produit pour le rendre téléchargeable depuis
Jenkins
                archiveArtifacts artifacts: 'target/*.jar', fingerprint: true
            }
        }
    }

    // -------------------------------------------------------
    // POST : actions exécutées à la fin du pipeline
    // -------------------------------------------------------
    post {
        success {
            echo "Pipeline terminé avec succès !"
        }
        failure {
            echo "Le pipeline a échoué. Consultez les logs."
        }
        always {
            // Nettoyer le workspace après chaque build
            cleanWs()
        }
    }
}
```

Les instructions Groovy essentielles

Voici les instructions les plus courantes que vous utiliserez dans vos Jenkinsfiles :

Instruction	Description
`sh 'commande'`	Exécuter une commande shell Linux.
`echo 'message'`	Afficher un message dans les logs.
`checkout scm`	Cloner le dépôt Git associé au job.
`git url: '...', branch: '...'`	Cloner un dépôt Git spécifique.
`archiveArtifacts 'target/*.jar'`	Archiver des fichiers produits par le build.
`junit 'target/surefire-reports/*.xml'`	Publier les résultats des tests JUnit.
`withCredentials([...])`	Injecter des secrets de manière sécurisée.
`parallel { stage(...) ... }`	Exécuter plusieurs stages simultanément.
`input message: '...'`	Mettre le pipeline en pause pour une validation humaine.
`cleanWs()`	Nettoyer le workspace après le build.

Les stages parallèles

Pour accélérer un pipeline, il est possible d'exécuter plusieurs stages en même temps grâce à l'instruction `parallel` :

```
stage('Vérifications') {
    parallel {
        stage('Tests Unitaires') {
            steps { sh 'mvn test -B' }
        }
        stage('Analyse SonarQube') {
            steps {
                withSonarQubeEnv('SonarQube') {
                    sh 'mvn sonar:sonar'
                }
            }
        }
    }
}
```

Les deux stages s'exécutent en même temps, réduisant la durée totale du pipeline.

> ***Référence TP :*** *Le TP 5 vous guide dans la création d'un pipeline CI/CD complet avec toutes ces étapes assemblées.*

---

5. Les Agents

Déclarer un agent dans le Jenkinsfile

L'instruction `agent` au niveau du bloc `pipeline` définit l'environnement d'exécution par défaut pour tout le pipeline. Elle peut aussi être déclarée au niveau d'un `stage` pour surcharger l'agent par défaut pour ce stage uniquement.

`agent any` : Jenkins utilise n'importe quel agent disponible.

```
pipeline {
    agent any
    // ...
}
```

`agent none` : Aucun agent global. Chaque stage doit déclarer son propre agent. Utile pour les pipelines multi-agents.

```
pipeline {
    agent none
    stages {
        stage('Build Java') {
            agent { docker { image 'maven:3.9-eclipse-temurin-17' } }
            steps { sh 'mvn package' }
        }
        stage('Build Node') {
            agent { docker { image 'node:20-alpine' } }
            steps { sh 'npm install && npm run build' }
        }
    }
}
```

`agent { label 'linux-agent' }` : Jenkins utilise uniquement les agents ayant le label `linux-agent`. Les labels permettent de cibler des agents spécifiques (ex: agents avec Docker installé, agents Windows…).

`agent { docker { image '...' } }` : Jenkins lance un conteneur Docker avec l'image spécifiée et exécute le build à l'intérieur. C'est la méthode recommandée pour les projets Java/Maven.

```
agent {
    docker {
        image 'maven:3.9-eclipse-temurin-17'
        // Monter le cache Maven local pour éviter de re-télécharger les
dépendances à chaque build
        args '-v /root/.m2:/root/.m2'
    }
}
```

Configurer un Agent permanent

Pour ajouter un Agent permanent à Jenkins :

1. Allez dans **Manage Jenkins > Nodes and Clouds > New Node**.
2. Donnez un nom à l'agent et choisissez **"Permanent Agent"**.
3. Configurez :
    - **Remote root directory** : Le dossier sur l'agent où Jenkins stockera ses fichiers (ex: `/home/jenkins`).

- **Labels** : Mots-clés pour cibler cet agent (ex: `linux docker`).
- **Launch method** : SSH (fournissez l'adresse IP, le port et les credentials SSH).

---

6. Les Outils Globaux

Pour que Jenkins puisse exécuter `mvn clean compile` ou `git clone`, il doit savoir où ces outils sont installés sur les agents.

Configuration dans Manage Jenkins > Tools

Vous pouvez configurer les outils suivants :

Outil	Description
JDK	Version de Java à utiliser pour les builds.
Maven	Chemin ou version de Maven à utiliser.
Git	Chemin vers l'exécutable Git sur les agents.
Docker	Chemin vers l'exécutable Docker.

Pour chaque outil, deux approches sont possibles :

1. **Chemin local :** Vous indiquez le chemin absolu de l'outil sur les agents (ex: `/usr/bin/mvn`). Cela suppose que l'outil est déjà installé.
2. **Installation automatique :** Jenkins télécharge et installe l'outil lui-même depuis internet lors du premier build. Pratique pour les nouvelles installations.

Note importante : Si vous utilisez des Agents Docker (comme dans les TPs), cette configuration devient souvent inutile. L'outil est déjà inclus dans l'image Docker utilisée comme agent. Par exemple, l'image `maven:3.9-eclipse-temurin-17` contient déjà Maven et le JDK 17.

---

7. Les Variables d'Environnement

Les variables d'environnement permettent de rendre vos pipelines dynamiques et réutilisables.

Variables intégrées (Built-in)

Jenkins fournit automatiquement des dizaines de variables utiles dans chaque build. Voici les plus courantes :

Variable	Description	Exemple de valeur
`BUILD_NUMBER`	Numéro incrémental du build.	`42`
`BUILD_URL`	URL directe vers les logs du build.	`http://jenkins:8080/job/mon-app/42/`
`WORKSPACE`	Chemin absolu du répertoire de travail sur l'agent.	`/var/jenkins/workspace/mon-app`
`BRANCH_NAME`	Nom de la branche Git en cours de build.	`feature/login`
`GIT_COMMIT`	Hash SHA-1 du commit en cours.	`a3f5c2d...`
`JOB_NAME`	Nom du job Jenkins.	`mon-app`

Pour voir toutes les variables disponibles, accédez à l'URL `http://<votre-jenkins>/env-vars.html` depuis votre navigateur.

Variables personnalisées

Vous pouvez définir vos propres variables dans le bloc `environment {}` du Jenkinsfile. Ces variables sont disponibles dans tous les stages suivants.

```
environment {
    APP_NAME    = 'demo-java-app'
    REGISTRY    = 'registry.example.com'
    // Combiner une valeur fixe et une variable built-in
    APP_VERSION = "1.0.${BUILD_NUMBER}"
}
```

Pour utiliser une variable dans un script shell, utilisez la syntaxe `${NOM_VARIABLE}` :

```
sh "docker build -t ${REGISTRY}/${APP_NAME}:${APP_VERSION} ."
```

Variables globales (System)

Dans **Manage Jenkins > System**, vous pouvez définir des variables d'environnement globales, disponibles dans **tous** les jobs de l'instance Jenkins. C'est utile pour des valeurs partagées comme l'URL d'un registre Docker ou d'un serveur Nexus.

---

8. Les Credentials

Pourquoi les Credentials sont-ils essentiels ?

Un pipeline a souvent besoin de s'authentifier auprès de systèmes externes : cloner un dépôt Git privé, pousser une image sur Docker Hub, envoyer une analyse à SonarQube, déployer sur un serveur…

Règle absolue : Ne mettez **jamais** de mots de passe, tokens ou clés en clair dans un Jenkinsfile. Le Jenkinsfile est stocké dans Git, accessible à tous les membres de l'équipe. Un secret en clair dans Git est un secret compromis.

Les types de Credentials

Jenkins supporte plusieurs types de secrets :

Type	Description	Cas d'usage
Secret text	Une chaîne de caractères secrète.	Token SonarQube, clé d'API.
Username with password	Un couple identifiant / mot de passe.	Docker Hub, GitLab, Nexus.
SSH Username with private key	Une clé SSH privée.	Connexion Git via SSH, déploiement sur serveur.
Secret file	Un fichier secret (ex: fichier de configuration).	Fichier kubeconfig Kubernetes.
Certificate	Un certificat PKCS#12.	Authentification par certificat.

Ajouter un Credential

1. Allez dans **Manage Jenkins > Credentials > System > Global credentials**.
2. Cliquez sur **"Add Credentials"**.
3. Choisissez le type, renseignez les valeurs et donnez un **ID unique** (ex: `sonar-token`, `docker-hub-creds`). Cet ID est ce que vous utiliserez dans vos Jenkinsfiles.

Utiliser un Credential dans un Jenkinsfile

Méthode 1 : `withCredentials` (recommandée pour les scripts shell)

```
stage('Push Docker') {
    steps {
        withCredentials([
            usernamePassword(
                credentialsId: 'docker-hub-creds',
                usernameVariable: 'DOCKER_USER',
                passwordVariable: 'DOCKER_PASS'
            )
        ]) {
            // Les variables DOCKER_USER et DOCKER_PASS sont disponibles ici
            // Jenkins masque automatiquement leur valeur dans les logs
(affichées comme ****)
            sh 'echo $DOCKER_PASS | docker login -u $DOCKER_USER --password-
stdin'
            sh 'docker push mon-image:latest'
        }
    }
}
```

Méthode 2 : `credentials()` dans le bloc `environment` (pour les tokens simples)

```
environment {
    // Injecte le credential 'sonar-token' dans la variable SONAR_TOKEN
    SONAR_TOKEN = credentials('sonar-token')
}
```

Méthode 3 : `withSonarQubeEnv` (spécifique à SonarQube)

```
stage('Analyse SonarQube') {
    steps {
        // Injecte automatiquement SONAR_HOST_URL et SONAR_TOKEN
        withSonarQubeEnv('SonarQube') {
            sh 'mvn sonar:sonar -Dsonar.projectKey=mon-projet'
        }
    }
}
```

> ***Référence TP :*** *Le TP 4 vous guide pas à pas dans la configuration d'un credential SonarQube et son utilisation dans un pipeline.*

---

9. Les Plugins

Rôle des plugins

Jenkins “nu” est un moteur d’automatisation générique. Toute sa puissance vient de ses **plugins** (plus de 1 800 disponibles). Chaque plugin ajoute une fonctionnalité spécifique : intégration avec Git, Docker, SonarQube, envoi d’emails, interface graphique…

Gérer les plugins

Dans **Manage Jenkins > Plugins**, vous trouverez quatre onglets :

Onglet	Description
Updates	Plugins installés pour lesquels une mise à jour est disponible.
Available plugins	Catalogue de tous les plugins disponibles à l’installation.
Installed plugins	Liste des plugins actuellement installés.
Advanced settings	Configuration du proxy, upload manuel d’un plugin (.hpi).

Pour installer un plugin, allez dans **Available plugins**, cherchez son nom, cochez-le et cliquez sur **“Install”**. Jenkins peut nécessiter un redémarrage après l’installation.

Les plugins incontournables

Plugin	Rôle
Git Plugin	Cloner des dépôts Git, déclencher des builds via webhook.
Pipeline	Support des Jenkinsfiles déclaratifs et scriptés.
Docker Pipeline	Utiliser Docker comme agent d'exécution dans les pipelines.
SonarQube Scanner	Lancer des analyses SonarQube et attendre le Quality Gate.
Maven Integration	Support natif de Maven dans les jobs Freestyle.
Blue Ocean	Interface graphique moderne pour visualiser les pipelines.
Credentials Binding	Injecter des credentials dans les variables d'environnement.
Email Extension	Envoyer des notifications email personnalisées.
Workspace Cleanup	Nettoyer automatiquement les workspaces (plugin `cleanWs()`).
GitLab Plugin	Intégration avancée avec GitLab (webhooks, statuts de MR).

---

10. Le Workspace et le Nettoyage

Qu'est-ce que le Workspace ?

Le **Workspace** (espace de travail) est le répertoire sur le disque dur de l'Agent où Jenkins :

1. Clone le code source depuis Git.
2. Exécute toutes les commandes du pipeline (compilation, tests…).
3. Produit les fichiers résultants (JARs, rapports de tests, images Docker…).

Le chemin du workspace est disponible via la variable `WORKSPACE`. Il ressemble généralement à `/var/jenkins_home/workspace/nom-du-job`.

Le piège du débutant

Par défaut, Jenkins **ne supprime pas** le workspace à la fin d'un build. Les fichiers compilés, les dépendances Maven téléchargées et les logs s'accumulent sur le disque de l'Agent.

Cela peut causer deux problèmes :

1. **Saturation du disque** : après des centaines de builds, le disque de l'Agent se remplit.
2. **Pollution entre builds** : un fichier résiduel d'un build précédent peut fausser le build suivant (ex: un ancien `.jar` qui n'est pas écrasé).

La solution : `cleanWs()`

La bonne pratique est de toujours nettoyer le workspace à la fin du build, en utilisant `cleanWs()` dans le bloc `post { always { ... } }` :

```
post {
    always {
        cleanWs()
    }
}
```

L'instruction `always` garantit que le nettoyage s'effectue que le build ait réussi ou échoué.

---

11. Les Notifications

Le bloc `post`

Le bloc `post` dans un Jenkinsfile permet de définir des actions à exécuter à la fin du pipeline, en fonction de son résultat. Les conditions disponibles sont :

Condition	Déclenchement
`always`	Toujours, quel que soit le résultat.
`success`	Uniquement si le pipeline a réussi.
`failure`	Uniquement si le pipeline a échoué.
`unstable`	Si le pipeline est instable (tests échoués mais build réussi).
`changed`	Si le statut a changé par rapport au build précédent (succès → échec ou vice-versa).
`aborted`	Si le pipeline a été annulé manuellement.

La condition `changed` est particulièrement utile pour les notifications : elle évite d'envoyer un email à chaque build et n'alerte que lors d'un changement d'état.

Exemple de notification email

```
post {
    failure {
        emailext(
            to: 'equipe-dev@sncf.fr',
            subject: "Build échoué : ${JOB_NAME} #${BUILD_NUMBER}",
            body: """
                Le build ${BUILD_URL} a échoué.
                Branche : ${BRANCH_NAME}
                Commit  : ${GIT_COMMIT}
                Consultez les logs pour plus de détails.
            """
        )
    }
    changed {
        echo "Le statut du build a changé !"
    }
    always {
        cleanWs()
    }
}
```

---

12. Bonnes pratiques et pièges courants

Les 8 règles à retenir

1. Ne jamais exécuter de builds sur le Controller. Toujours utiliser des Agents. Si vous démarrez Jenkins pour des TPs sur une seule machine, configurez un agent local ou utilisez des agents Docker.

2. Ne jamais mettre de secrets en clair dans le Jenkinsfile. Utilisez toujours le Credentials Store de Jenkins et les fonctions `withCredentials()` ou `credentials()`.

3. Toujours nettoyer le workspace. Ajoutez `cleanWs()` dans le bloc `post { always { ... } }` de chaque pipeline.

4. Versionner le Jenkinsfile avec le code. Le `Jenkinsfile` doit être à la racine du dépôt Git. Ne jamais le stocker uniquement dans Jenkins.

5. Utiliser des agents Docker pour les builds. Les agents Docker garantissent un environnement propre et reproductible à chaque build, éliminant le problème “ça marche sur ma machine”.

6. Préférer les Pipelines aux Jobs Freestyle. Les Pipelines sont versionnables, maintenables et bien plus puissants. Les Jobs Freestyle ne sont adaptés qu’aux tâches très simples.

7. Garder les pipelines rapides. Un pipeline CI doit s’exécuter en moins de 10 minutes. Utilisez les stages parallèles, le cache Maven (`-v /root/.m2:/root/.m2`) et des images Docker légères.

8. Mettre les plugins à jour régulièrement. Les mises à jour de plugins corrigent souvent des failles de sécurité. Vérifiez régulièrement l’onglet “Updates” dans la gestion des plugins.

---

13. FAQ Débutant

Q : Mon build fonctionne sur mon PC, mais échoue sur Jenkins. Pourquoi ?

Jenkins exécute le build dans un environnement vierge (le Workspace de l'Agent). Sur votre PC, vous avez peut-être des variables d'environnement locales, des dépendances déjà en cache (`~/.m2`), ou une version de Java différente. L'utilisation d'Agents Docker résout ce problème en garantissant un environnement strictement identique à chaque build.

Q : Comment trouver la cause d'un build en échec (boule rouge) ?

Cliquez sur le numéro du build en échec (ex: `#42`), puis cliquez sur **"Console Output"** (ou "Logs"). Vous verrez le texte complet généré par le build. Remontez depuis le bas de la page en cherchant les mots `ERROR` ou `FAILURE` pour trouver la ligne exacte qui a causé l'échec.

Q : J'ai une faute de frappe dans mon Jenkinsfile. Comment Jenkins réagit-il ?

Jenkins va échouer très rapidement dès la lecture du fichier, avec une erreur de type `org.codehaus.groovy.control.MultipleCompilationErrorsException`. Il n'exécutera aucune étape. Les logs indiqueront précisément la ligne et la nature de l'erreur de syntaxe.

Q : Faut-il installer Docker sur le Controller ou sur les Agents ?

Docker doit être installé sur la machine qui exécute physiquement les builds, c'est-à-dire l'**Agent**. Si vous utilisez un conteneur Jenkins "tout-en-un" pour des TPs (où le Controller fait aussi office d'Agent), il faut monter le socket Docker (`/var/run/docker.sock`) dans le conteneur Jenkins pour lui donner accès au Docker de la machine hôte. C'est exactement ce que fait la commande du TP 1 :

```
-v /var/run/docker.sock:/var/run/docker.sock
```

Q : Quelle est la différence entre un build "Failure" (rouge) et "Unstable" (jaune) ?

- **Failure (rouge) :** Le build a rencontré une erreur bloquante. La compilation a échoué, une commande shell a retourné un code d'erreur non nul, ou une exception non gérée s'est produite.

- **Unstable (jaune) :** Le build s'est terminé, mais avec des avertissements. Le cas le plus courant : la compilation a réussi, mais certains tests JUnit ont échoué. Jenkins considère que le code "tourne" mais n'est pas fiable.

Q : Peut-on avoir plusieurs Jenkinsfiles dans un même dépôt ?

Par convention, un seul `Jenkinsfile` est placé à la racine du dépôt. Cependant, il est possible de spécifier un chemin différent dans la configuration du job Jenkins (champ "Script Path"). Cela peut être utile dans un dépôt mono-repo contenant plusieurs projets.

Q : Qu'est-ce que le "Quality Gate" de SonarQube ?

Un Quality Gate est un ensemble de seuils de qualité que le code doit respecter. Par exemple : "0 nouveaux bugs", "couverture de tests >= 80%". Dans Jenkins, l'instruction `waitForQualityGate abortPipeline: true` met le pipeline en pause et attend le résultat de l'analyse SonarQube. Si les seuils ne sont pas respectés, le pipeline échoue automatiquement, empêchant le déploiement d'un code de mauvaise qualité. Voir le TP 4 pour la mise en pratique.

Q : Comment savoir quels plugins sont installés sur mon Jenkins ?

Allez dans **Manage Jenkins > Plugins > Installed plugins**. Vous verrez la liste complète avec les versions. Vous pouvez également utiliser l'API Jenkins : `http://<votre-jenkins>/pluginManager/api/json?depth=1`.

Q : Peut-on annuler un build en cours ?

Oui. Dans la vue du job, cliquez sur le build en cours (indiqué par une barre de progression animée), puis cliquez sur le bouton rouge **"Abort"** (ou l'icône ×). Le build sera marqué comme "Aborted" (gris).

Ce document est un support de formation. Pour aller plus loin, consultez la documentation officielle de Jenkins : https://www.jenkins.io/doc/